计算机系统CSAPP课后作业答案

计科210X 甘晴void 202108010XXX

第2章

2.61

**2.61 写一个C表达式，在下列描述的条件下产生1，而在其他情况下得到0。假设x是int类型。
A. x的任何位都等于1。
B. x的任何位都等于0。
C. x的最高有效字节中的位都等于1。
D. x的最低有效字节中的位都等于0。
代码应该遵循位级整数编码规则，另外还有一个限制，你不能使用相等（==）和不相等（!=）测试。

解：

(!~x) || (!x) || (!~(x|0x00ffffff)) || (!(x&0x000000ff))

或者：

(!~x) || (!x) || (!~(x>>24)) || (!(x<<24))

2.71

*2.71 你刚刚开始在一家公司工作，他们要实现一组过程来操作一个数据结构，要将4个有符号字节封装成一个32位unsigned。一个字中的字节从0（最低有效字节）编号到3（最高有效字节）。分配给你的任务是：为一个使用补码运算和算术右移的机器编写一个具有如下原型的函数：

```
/* Declaration of data type where 4 bytes are packed
   into an unsigned */
typedef unsigned packed_t;

/* Extract byte from word.  Return as signed integer */
int xbyte(packed_t word, int bytenum);
```

也就是说，函数会抽取出指定的字节，再把它符号扩展为一个32位int。
你的前任（因为水平不够高而被解雇了）编写了下面的代码：

```
/* Failed attempt at xbyte */
int xbyte(packed_t word, int bytenum)
{
    return (word >> (bytenum << 3)) & 0xFF;
}
```

A. 这段代码错在哪里？
B. 给出函数的正确实现，只能使用左右移位和一个减法。

A.

实现的是逻辑位移，扩展后前面全是0，不符合符号扩展的要求

B.

```
int xbyte(packed_t word,int bytenum)
{
  return ((int)word<<((3-bytenum)<<3))>>24;
}
```

2.87

1. 格式 A
■ 有一个符号位。
■ 有 $k=5$ 个阶码位。阶码偏置量是 15。
■ 有 $n=3$ 个小数位。
2. 格式 B
■ 有一个符号位。
■ 有 $k=4$ 个阶码位。阶码偏置量是 7。
■ 有 $n=4$ 个小数位。
下面给出了一些格式 A 表示的位模式，你的任务是把它们转换成最接近的格式 B 表示的值。如果需要舍入，你要向 $+\infty$ 舍入。另外，给出用格式 A 和格式 B 表示的位模式对应的值。要么是整数（例如，17），要么是小数（例如，17/64 或 $17/2^6$）。

格式A		格式B	
位	值	位	值
1 01110 001	$\frac{-9}{16}$	1 0110 0010	$\frac{-9}{16}$
0 10110 101			
1 00111 110			
0 00000 101			
1 11011 000			
0 11000 100			

格式**A**		格式**B**	
位	值	位	值
1 01110 001	-9/16	1 0110 0010	-9/16
0 10110 101	208	0 1110 1010	208
1 00111 110	-7/2^10	1 0000 0111	-7/2^10
0 00000 101	5/2^17	0 0000 0001	1/2^10
1 11011 000	-2^12	1 1111 0000	-inf
0 11000 100	768	0 1111 0000	+inf

2.88

*2.88 我们在一个 int 类型为 32 位补码表示的机器上运行程序。float 类型的值使用 32 位 IEEE 格式，而 double 类型的值使用 64 位 IEEE 格式。

我们产生随机整数 x、y 和 z，并且把它们转换成 double 类型的值：

```
/* Create some arbitrary values */
int x = random();
int y = random();
int z = random();
/* Convert to double */
double   dx = (double) x;
double   dy = (double) y;
double   dz = (double) z;
```

对于下列的每个 C 表达式，你要指出表达式是否总是为 1。如果它总是为 1，描述其中的数学原理。否则，列举出使它为 0 的参数的例子。请注意，不能使用 IA32 机器运行 GCC 来测试你的答案，因为对于 float 和 double，它使用的都是 80 位的扩展精度表示。

A. `(double)(float) x == dx`

B. `dx + dy == (double) (x+y)`

C. `dx + dy + dz == dz + dy + dx`

D. `dx * dy * dz == dz * dy * dx`

E. `dx / dx == dy / dy`

前置知识：

在IEEE754下，

32位包括符号位1位，阶码8位，尾数23位，其阶码偏置量为127；

64位包括符号位1位，阶码11位，尾数52位，其阶码偏置量为1023；

下面的讨论都在上述条件下进行

A. 不总是为1；

该式等价于(double)(float)x==(double)x，但因为将x转换为double和float时有可能发生float无法精确表示该int而double可以精确表示该int的情况，而从float到double这一步不会发生问题，可以精确表示。所以可能等式不成立，下面给出反例。

反例：取x=67,108,863

其64位表示为0 10000011000 11111111111111111111111110.....

（尾数有25个1，其阶码E=1048=1023+25，尾数M=1.11.....(小数点后25个1)）



而对其进行32位表示时由于32位的尾数只有23位，无法精确表示该数，所以会发生舍入，导致(float)x！=(double)x

B. 不总是为1；

该式等价于(double)x+(double)y==(double)(x+y)

反例：取x=y=(1.11……1)*2^1023，即所能表示最大的数，此时右边溢出，等式不成立

C. 总是为1；

由加法交换律。即使溢出，其溢出值也相同，等式左右相等。

D. 不总是为1；

从一个角度说，若溢出，其溢出值可能不同，等式左右不一定相等。

从另一个角度，double无法精确表示2^64以内的所有整数，所以在第一次乘法后就已经开始产生误差了。

E. 不总是为1；

反例：取x=0，y≠0，有左边=inf≠右边

第3章

3.34

练习题 3.34 一个具有通用结构的 C 函数如下：

```
int rfun(unsigned x) {
    if ( _________ )
        return _________;
    unsigned nx = _________;
    int rv = rfun(nx);
    return _________;
}
```

GCC 产生如下汇编代码（省略了建立和完成代码）：

```
1     movl    8(%ebp), %ebx
2     movl    $0, %eax
3     testl   %ebx, %ebx
4     je      .L3
5     movl    %ebx, %eax
6     shrl    %eax              Shift right by 1
7     movl    %eax, (%esp)
8     call    rfun
9     movl    %ebx, %edx
10    andl    $1, %edx
11    leal    (%edx,%eax), %eax
12  .L3:
```

A. rfun 存储在被调用者保存寄存器 %ebx 中的值是什么？
B. 填写上述 C 代码中缺失的表达式。
C. 用自然语言描述这段代码计算的功能。

A.

是调用者接收到的x（也就是未被右移过的x）

B.

```
int rfun(unsigned x)
{
  if (x == 0)
     return 0;
  unsigned nx = x >> 1;
  int rv = rfun(nx);
  return ((x & 0x1) + rv);
}
```

C.

递归计算二进制数x中有多少个1（或表述为计算参数x中位的和）

3.56

**3.56 考虑下面的汇编代码：

```
  x at %ebp+8, n at %ebp+12
1     movl    8(%ebp), %esi
2     movl    12(%ebp), %ebx
3     movl    $1431655765, %edi
4     movl    $-2147483648, %edx
5   .L2:
6     movl    %edx, %eax
7     andl    %esi, %eax
8     xorl    %eax, %edi
9     movl    %ebx, %ecx
10    shrl    %cl, %edx

11    testl   %edx, %edx
12    jne     .L2
13    movl    %edi, %eax
```

以上代码是以下整体形式的C代码编译产生的：

```
1   int loop(int x, int n)
2   {
3       int result = _____;
4       int mask;
5       for (mask = _____; mask  _____ ; mask = _____ ) {
6           result ^= _____ ;
7       }
8       return result;
9   }
```

你的任务是填写这个C代码中缺失的部分，得到一个程序等价于产生的汇编代码。回想一下，这个函数的结果是在寄存器%eax中返回的。你会发现以下工作很有帮助：检查循环之前、之中和之后的汇编代码，形成一个寄存器和程序变量之间一致的映射。

A. 哪个寄存器保存着程序值x、n、result和mask？

B. result和mask的初始值是什么？

C. mask的测试条件是什么？

D. mask是如何被修改的？

E. result是如何被修改的？

F. 填写这段C代码中所有缺失的部分。

A.

%esi保存x

%ebx保存n

%edi保存result

%edx保存mask

B.

Result初值为0x55555555

Mask初值为0x80000000

C.

测试条件为mask是否为0

D.

Mask右移（n&0x000000ff）位

E.

Result^= (mask & x)

F.代码如下

```
int loop(int x, int n)
{
  int result = 0x55555555;
  int mask;
  for (mask = 0x80000000; mask != 0; mask = ((unsigned)mask) >> (n
& 0x000000ff))
  {
     result ^= (mask & x);
  }
  return result;
}
```

3.59

**3.59 这个程序给你一个机会，逆向工程一个 switch 语句。在下面这个过程中，去掉了 switch 语句的主体：

```
1  int switch_prob(int x, int n)
2  {
3      int result = x;
4
5      switch(n) {
6          /* Fill in code here */
7      }
8      return result;
9  }
```

图 3-44 给出了这个过程的反汇编机器代码。我们可以看到，在第 4 行，参数 n 被加载到寄存器 %eax 中。

```
1   08048420 <switch_prob>:
2    8048420:  55                    push   %ebp
3    8048421:  89 e5                 mov    %esp,%ebp
4    8048423:  8b 45 0c              mov    0xc(%ebp),%eax
5    8048426:  83 e8 28              sub    $0x28,%eax
6    8048429:  83 f8 05              cmp    $0x5,%eax
7    804842c:  77 07                 ja     8048435 <switch_prob+0x15>
8    804842e:  ff 24 85 f0 85 04 08  jmp    *0x80485f0(,%eax,4)
9    8048435:  8b 45 08              mov    0x8(%ebp),%eax
10   8048438:  eb 24                 jmp    804845e <switch_prob+0x3e>
11   804843a:  8b 45 08              mov    0x8(%ebp),%eax
12   804843d:  8d 76 00              lea    0x0(%esi),%esi
13   8048440:  eb 19                 jmp    804845b <switch_prob+0x3b>
14   8048442:  8b 45 08              mov    0x8(%ebp),%eax
15   8048445:  c1 e0 03              shl    $0x3,%eax
16   8048448:  eb 17                 jmp    8048461 <switch_prob+0x41>
17   804844a:  8b 45 08              mov    0x8(%ebp),%eax
18   804844d:  c1 f8 03              sar    $0x3,%eax
19   8048450:  eb 0f                 jmp    8048461 <switch_prob+0x41>
20   8048452:  8b 45 08              mov    0x8(%ebp),%eax
21   8048455:  c1 e0 03              shl    $0x3,%eax
22   8048458:  2b 45 08              sub    0x8(%ebp),%eax
23   804845b:  0f af c0              imul   %eax,%eax
24   804845e:  83 c0 11              add    $0x11,%eax
25   8048461:  5d                    pop    %ebp
26   8048462:  c3                    ret
```

图 3-44　家庭作业 3.59 的反汇编代码

跳转表驻留在另一个存储器区域中。可以从第 8 行的间接跳转看出来，跳转表的起始地址为 0x80485f0。用调试器 GDB，我们可以用命令 x/6w 0x80485f0 来检查存储器中的 6 个 4 字节的字。GDB 打印出下面的内容：

```
(gdb) x/6w 0x80485f0
0x80485f0: 0x08048442 0x08048435 0x08048442 0x0804844a
0x8048600: 0x08048452 0x0804843a
```

用 C 代码填写开关语句的主体，使它的行为与机器代码一致。

补全代码如下：

```
int switch_prob(int x, int n)
{
  int result = x;
  switch (n)
  {
  case 40:
     result <<= 3;
```

```
      break;
   case 41:
      result += 0x11;
      break;
   case 42:
      result <<= 3;
      break;
   case 43:
      result = ((unsigned)result) >> 3;
      break;
   case 44:
      result = ((result << 3) - x);
      result *= result;
      result += 0x11;
      break;
   case 45:
      result *= result;
      result += 0x11;
      break;
   default:
      result += 0x11;
      break;
   }
   return result;
}
```

3.66

*** 3.66 你负责维护一个大型的 C 程序时，遇到下面这样的代码：

```
typedef struct {
    int left;
    a_struct a[CNT];
    int right;
} b_struct;

void test(int i, b_struct *bp)
{
    int n = bp->left + bp->right;
    a_struct *ap = &bp->a[i];
    ap->x[ap->idx] = n;
}
```

编译时常数 CNT 和结构 a_struct 的声明在一个你没有访问权限的文件中。幸好，你有代码的 '.o' 版本，可以用 OBJDUMP 程序来反汇编这些文件，得到如图 3-45 所示的反汇编代码。

```
00000000 <test>:
   0:   55                      push   %ebp
   1:   89 e5                   mov    %esp,%ebp
   3:   53                      push   %ebx
   4:   8b 45 08                mov    0x8(%ebp),%eax
   7:   8b 4d 0c                mov    0xc(%ebp),%ecx
   a:   6b d8 1c                imul   $0x1c,%eax,%ebx
   d:   8d 14 c5 00 00 00 00    lea    0x0(,%eax,8),%edx
  14:   29 c2                   sub    %eax,%edx
  16:   03 54 19 04             add    0x4(%ecx,%ebx,1),%edx
  1a:   8b 81 c8 00 00 00       mov    0xc8(%ecx),%eax
  20:   03 01                   add    (%ecx),%eax
  22:   89 44 91 08             mov    %eax,0x8(%ecx,%edx,4)
  26:   5b                      pop    %ebx
  27:   5d                      pop    %ebp
  28:   c3                      ret
```

图 3-45 家庭作业 3.66 的反汇编代码

运用你的逆向工程技术，推断出下列内容：

A. CNT 的值。

B. 结构 a_struct 的完整声明。假设这个结构中只有字段 idx 和 x。

解：

先写答案

A.CNT=7

B.完整声明如下

```
typedef struct
{
  int idx;
  int x[6];
} a_struct;
```

这题有点意思，我还想写一下*解题的过程*。

第13行的mov将%eax移到一个地址，此时的%eax里应该是计算出的n，所以结合第11和第12行，发现这两行实际上算的是int n = bp->left + bp->right，对应bp->left地址就是一开始存在%ecx中的(bp)，而bp->right是(bp+0xc8)，也就是(bp+200)，所以可以推断a_struct[CNT]共占196字节。（这里bp表示0xc%ebp，为方便简写为bp）

然后我们可以分析含信息量最多的第10行，第9行我们可以看到%edx内是7i。来到第10行，%edx内存入的是【(bp+4+28i)所在内存的值+7i】，这里为什么加上7i有点费解，我留在后面探究，但是(bp+4+28i)应该是比较好看懂的，这个应该就是a[i]所在的地址，由此我们知道a_struct一个应该占28字节，用196÷28=7，可以得知CNT=7。

而且其实从这里也很好看出既然取的是(bp+4+28i)，说明这应该就是idx的地址，也就是说在a_struct内部，idx先于x被定义。这一点后面还有验证。

最后我们重新分析第13行，这次我们分析mov的终点。是0x8(%ecx,%edx,4)，也就是(4%edx+bp+8)，由上面我们知道%dex内是什么，代入后，实际上mov的终点是(bp+4+28i+4+4*【(bp+4+28i)所在内存的值】)。

现在一切都清晰了！！“bp+4+28i”定位到a[i]；“+4”是因为有idx是4字节，所以从这里可以验证idx应该在结构中定义在前面；“+4*【(bp+4+28i)所在内存的值】”定位到bp->a[i]->x[bp->a[i]->idx]，也就是ap->x[ap->idx]。

再回到解题上，一个a_struct共28字节，int型的x占4字节，剩下24字节应该就是一个6位int数组，故int x[6]。

至此这道题应该就完全理顺了。

第5章

5.19

**5.19 库函数 memset 的原型如下：

```
void *memset(void *s, int c, size_t n);
```

这个函数将从 s 开始的 n 个字节的存储器区域都填充为 c 的低位字节。例如，通过将参数 c 设置为 0，可以用这个函数来对一个存储器区域清零，不过用其他值也是可以的。
下面是 memset 最直接的实现：

```
1   /* Basic implementation of memset */
2   void *basic_memset(void *s, int c, size_t n)
3   {
4       size_t cnt = 0;
5       unsigned char *schar = s;
6       while (cnt < n) {
7           *schar++ = (unsigned char) c;
8           cnt++;
9       }
10      return s;
11  }
```

实现该函数一个更有效的版本，使用数据类型为 unsigned long 的字来装下 4 个（对于 IA32）或者 8 个（对于 x86-64）个 c，然后用字级的写遍历目标存储器区域。你可能发现增加额外的循环展开会有所帮助。在 Intel Core i7 机器上，我们能够把 CPE 从直接实现的 2.00 降低到对于 IA32 为 0.25，和对 x86-64 为 0.125，每个周期写 4 个或者 8 个字节。
这里是一些额外的指导原则。在此，假设 K 表示你运行程序的机器上的 sizeof(unsigned long) 的值。

- 你不可以调用任何库函数。
- 你的代码应该对任意 n 的值都能工作，包括当它不是 K 的倍数的时候。你可以用类似于使用循环展开时完成最后几次迭代的方法做到这一点。
- 你写的代码应该做到无论 K 的值是多少，都能够正确编译和运行。使用操作 sizeof 来做到这一点。
- 在某些机器上，未对齐的写可能比对齐的写慢很多。（在某些非 x86 机器上，未对齐的写甚至可能会导致段错误。）写出这样的代码，开始时直到目的地址是 K 的倍数时，使用字节级的写，然后进行字级的写，（如果需要）最后采用用字节级的写。
- 注意 cnt 足够小以至于一些循环上界变成负数的情况。对于涉及 sizeof 运算符的表达式，可以用无符号运算来执行测试。（参见 2.2.8 节和家庭作业 2.72。）

```c
#include <bits/stdc++.h>

void *my_memset(void *s, int c, size_t n)
{
    size_t K = sizeof(unsigned long);
    size_t cnt = 0;

    // 开始部分进行字节级的写直到对齐
    unsigned char *schar = (unsigned char *)s;
    while ((size_t)schar % K != 0 && cnt < n)
    {
        *schar++ = (unsigned char)c;
        cnt++;
    }

    // 合成longc满足K个字节，每个字节都是c的低位
    unsigned long longc;
    unsigned char *temp = (unsigned char *)&longc;
```

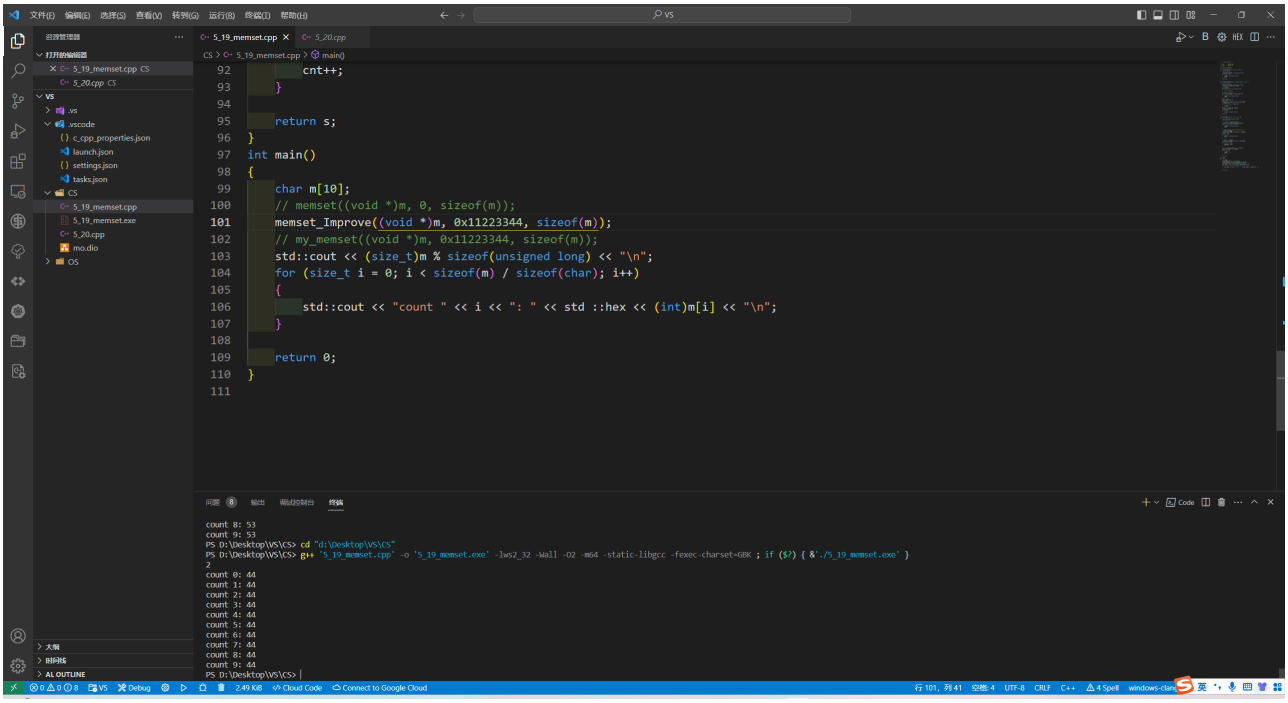
```
    size_t i = 0;
    while (i < K)
    {
        *temp++ = (unsigned char)c;
        i++;
    }

    // 对齐后进行字级的写
    unsigned long *slong = (unsigned long *)schar;
    while (cnt + K < n)
    {
        *slong++ = longc;
        cnt += K;
    }

    // 结尾部分可能的未成字部分进行字节级的写
    schar = (unsigned char *)slong;
    while (cnt < n)
    {
        *schar++ = c;
        cnt++;
    }

    return s;
}
int main()
{
    char m[10];
    my_memset((void *)m, 0x11223344, sizeof(m));
    std::cout << (size_t)m % sizeof(unsigned long) << "\n";
    for (size_t i = 0; i < sizeof(m) / sizeof(char); i++)
    {
        std::cout << "count " << i << ": " << std ::hex <<
(int)m[i] << "\n";
    }
    return 0;
}
```

测试结果：



5.20

5.20 在练习题 5.5 和 5.6 中我们考虑了多项式求值的任务，既有直接求值，也有用 Horner 方法求值。试着用我们讲过的优化技术写出这个函数更快的版本，这些技术包括循环展开、并行累积和重新结合。你会发现有很多不同的方法可以将 Horner 方法和直接求值与这些优化技术混合起来。

理想状况下，你能达到的 CPE 应该接近于你的机器上连续浮点加法和乘法之间的周期数（通常是 1）。至少，你应该能够达到一个小于你的机器的浮点加法延迟的 CPE。

```
double Polynomial_summation(double a[], double x, long degree)
{
    long i;
    double sum0 = 0, sum1 = 0;
    double xpwr0 = 1, xpwr1 = x;
    double x_step = x * x;
    // 循环展开
    for (i = 0; i < degree - 2; i += 2)
    {
        // 二路并行
        sum0 = sum0 + a[i] * xpwr0;
        sum1 = sum1 + a[i + 1] * xpwr1;

        xpwr0 *= x_step;
        xpwr1 *= x_step;
    }
```

```
    // 补充完整剩余的部分
    double sum_rest = 0;
    for (; i < degree; i++)
    {
        sum_rest = sum_rest + a[i] * xpwr0;
    }
    return sum0 + sum1 + sum_rest;
}
```

第6章

6.1

**6.42 你正在编写一个新的 3D 游戏，希望能名利双收。你现在正在写一个函数，使得在画下一帧之前先清空屏幕缓冲区。你现在工作的屏幕是 640×480 像素数组。你工作的机器有一个 64KB 直接映射高速缓存，每行 4 个字节。你使用下面的 C 语言数据结构：

```
1   struct pixel {
2       char r;
3       char g;
4       char b;
5       char a;
6   };
7
8   struct pixel buffer[480][640];
9   int i, j;
10  char *cptr;
11  int *iptr;
```

有如下假设：

- sizeof(char)==1 和 sizeof(int)==4。
- buffer 起始于存储器地址 0。
- 高速缓存初始为空。
- 唯一的存储器访问是对于 buffer 数组中元素的访问。变量 i、j、cptr 和 iptr 被存放在寄存器中。

下面代码中百分之多少的写会在高速缓存中不命中？

```
1       for (j = 0; j < 640; j++) {
2           for (i = 0; i < 480; i++){
3               buffer[i][j].r = 0;
4               buffer[i][j].g = 0;
5               buffer[i][j].b = 0;
6               buffer[i][j].a = 0;
7           }
8       }
```

25%

6.2

**6.43 给定家庭作业 6.42 中的假设，下面代码中百分之多少的写会在高速缓存中不命中？

```
1       char *cptr = (char *) buffer;
2       for (; cptr < (((char *) buffer) + 640 * 480 * 4); cptr++)
3           *cptr = 0;
```

25%

6.3

**6.44 给定家庭作业 6.42 中的假设，下面代码中百分之多少的写会在高速缓存中不命中？

```
1       int *iptr = (int *)buffer;
2       for (; iptr < ((int *)buffer + 640*480); iptr++)
3           *iptr = 0;
```

25%

6.4

***6.46 在这项任务中，你会把你在第 5 章和第 6 章中学习到的概念应用到一个存储器使用频繁的代码的优化问题上。考虑一个拷贝并转置一个类型为 int 的 $N \times N$ 矩阵的过程。也就是，对于源矩阵 S 和目的矩阵 D，我们要将每个元素 $s_{i,j}$ 拷贝到 $d_{j,i}$。只用一个简单的循环就能实现这段代码：

```
1   void transpose(int *dst, int *src, int dim)
2   {
3       int i, j;
4
5       for (i = 0; i < dim; i++)
6           for (j = 0; j < dim; j++)
7               dst[j*dim + i] = src[i*dim + j];
8   }
```

这里，过程的参数是指向目的矩阵（dst）和源矩阵（src）的指针，以及矩阵的大小 N（dim）。你的工作是设计一个运行得尽可能快的转置函数。

假设数据块的宽度为B，其生成由cache的容量C绝决定且有2*B^2<C,在此情况下B取最大值。

假设数据块的宽度为B，即每次读入B个数据，其生成由cache的容量C决定且有2*B^2<=C,在此情况下B取最大值。
以下为尝试的代码实现

```
#define B width_of_Block //2*B^2<=C
void transpose_AsSoonAsPossible(int *dst, int *src, int dim)
```

```
{
    long border=dim*dim;
    for ( int i=0; i<dim; i+=B; )
    {
        for ( int j=0; j<dim; j+=B; )
        {
            //确定一次作用的数据块的起始位置（即块的左上角的那个位置）
            for ( int m=i; m<i+B; m++)
            {
                for ( int n=j; n<j+B; n++)
                {
                    //对数据块的每一个位置进行操作
                    int dst_pos = n*dim + m;
                    int src_pos = m*dim + n;
                    if ( dst_pos<border && src_pos<border)
                     //必须保证不能超出边界（矩阵不一定是阵）
                    {
                        dst[dst_pos]=src[src_pos];
                    }
                }
            }
        }
    }
}
```

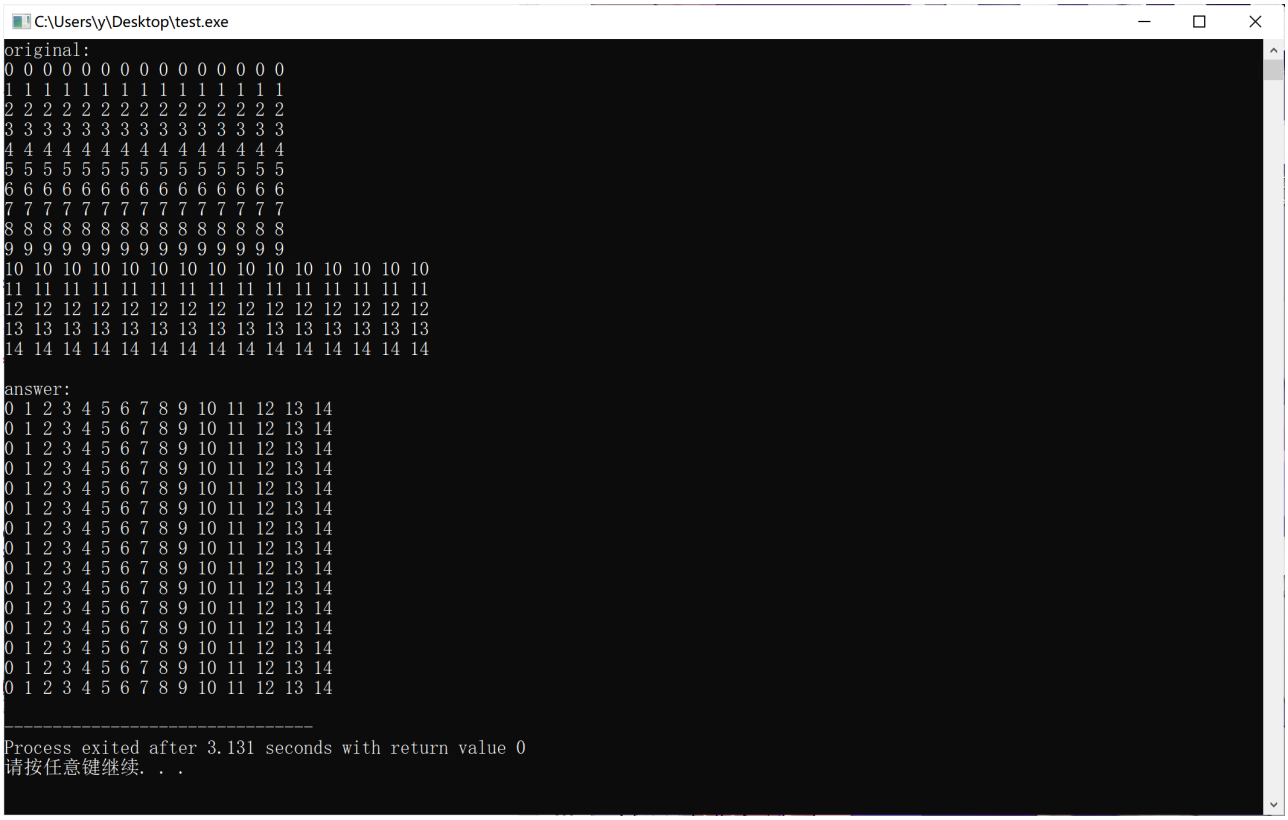
验证程序：

```
// 假设数据块的宽度为B，即每次读入B个数据，其生成由cache的容量C决定且有2 *B ^ 2
<= C, 在此情况下B取最大值。
// 下为尝试的代码实现
#define width_of_Block 4
#define B width_of_Block // 2*B^2<=C
void transpose_AsSoonAsPossible(int *dst, int *src, int dim)
{
    long border = dim * dim;
    for (int i = 0; i < dim; i += B)
    {
            for (int j = 0; j < dim; j += B)
        {
                    //确定一次作用的数据块的起始位置（即块的左上角的那个位
置）
```

```
            for (int m = i; m < i + B; m++)
            {
                for (int n = j; n < j + B; n++)
                {
                        //对数据块的每一个位置进行操作
                    int dst_pos = n * dim + m;
                    int src_pos = m * dim + n;
                    if (dst_pos < border && src_pos < border)
                        //必须保证不能超出边界
                    {
                        dst[dst_pos] = src[src_pos];
                    }
                }
            }
        }
    }
}

int main()
{
    int N=15;
    int *src=malloc(sizeof(int)*N*N);
    //initialization
    for (int i=0;i<N;i++)
        for (int j=0;j<N;j++)
                src[i*N+j]=i;
    printf("original:\n");
    for (int i=0;i<N;i++)
    {
        for (int j=0;j<N;j++)
                printf("%d ",src[i*N+j]);
        printf("\n");
    }
    int *dst=malloc(sizeof(int)*N*N);
    transpose_AsSoonAsPossible(dst, src, N);
    printf("\nanswer:\n");
    for (int i=0;i<N;i++)
    {
        for (int j=0;j<N;j++)
                printf("%d ",dst[i*N+j]);
        printf("\n");
    }
```

```
    free(dst);free(src);
}
```



第7章

7.12

**7.12 图 7-10 中的 swap 程序包含 5 个重定位的引用。对于每个重定位的引用，给出它在图 7-10 中的行号、运行时存储器地址和值。swap.o 模块中的原始代码和重定位条目如图 7-19 所示。

```
1  00000000 <swap>:
2     0:   55                      push   %ebp
3     1:   8b 15 00 00 00 00       mov    0x0,%edx        Get *bufp0=&buf[0]
4                                  3: R_386_32    bufp0   Relocation entry
5     7:   a1 04 00 00 00          mov    0x4,%eax        Get buf[1]
6                                  8: R_386_32    buf     Relocation entry
7     c:   89 e5                   mov    %esp,%ebp
8     e:   c7 05 00 00 00 00 04    movl   $0x4,0x0        bufp1 = &buf[1];
9    15:   00 00 00
10                                 10: R_386_32   bufp1   Relocation entry
11                                 14: R_386_32   buf     Relocation entry
12    18:   89 ec                  mov    %ebp,%esp
13    1a:   8b 0a                  mov    (%edx),%ecx     temp = buf[0];
14    1c:   89 02                  mov    %eax,(%edx)     buf[0]=buf[1];
15    1e:   a1 00 00 00 00         mov    0x0,%eax        Get *bufp1=&buf[1]
16                                 1f: R_386_32   bufp1   Relocation entry
17    23:   89 08                  mov    %ecx,(%eax)     buf[1]=temp;
18    25:   5d                     pop    %ebp
19    26:   c3                     ret
```

图 7-19 练习题 7.12 的代码和重定位条目

图 7-10 中的行号	地址	值

图7-10中的行号	地址	值
15	0x80483cb	0x804945c
16	0x80483d0	0x8049458
18	0x80483d8	0x8049548
18	0x80483dc	0x8049458
23	0x80483e7	0x8049548

7.13

的重定位条目中的信息：节偏移、重定位类型和符号名字。
可以随意使用诸如 OBJDUMP 之类的工具来帮助你解答这个题目。

```
1   extern int p3(void);
2   int x = 1;
3   int *xp = &x;
4
5   void p2(int y) {
6   }
7
8   void p1() {
9       p2(*xp + p3());
10  }
```

a）C 代码

```
1   00000000 <p2>:
2      0:   55                      push   %ebp
3      1:   89 e5                   mov    %esp,%ebp
4      3:   89 ec                   mov    %ebp,%esp
5      5:   5d                      pop    %ebp
6      6:   c3                      ret

7   00000008 <p1>:
8      8:   55                      push   %ebp
9      9:   89 e5                   mov    %esp,%ebp
10     b:   83 ec 08                sub    $0x8,%esp
11     e:   83 c4 f4                add    $0xfffffff4,%esp
12    11:   e8 fc ff ff ff          call   12 <p1+0xa>
13    16:   89 c2                   mov    %eax,%edx
14    18:   a1 00 00 00 00          mov    0x0,%eax
15    1d:   03 10                   add    (%eax),%edx
16    1f:   52                      push   %edx
17    20:   e8 fc ff ff ff          call   21 <p1+0x19>
18    25:   89 ec                   mov    %ebp,%esp
19    27:   5d                      pop    %ebp
20    28:   c3                      ret
```

b）可重定位目标文件的 .text 节

```
1   00000000 <x>:
2      0:   01 00 00 00
3   00000004 <xp>:
4      4:   00 00 00 00
```

c）可重定位目标文件的 .data 节

图 7-20　练习题 7.13 的示例代码

请注意：此题应在你自己的（虚拟）机器的Ubuntu上运行，依据你自己获得的可执行文件和可重定位的目标文件，进行解答。请说明你的解答过程。

> **我的答案：**

代码如下

```
extern int ps(void);
int x=1;
int *xp=&x;
void p2(int y){
}
void p1(){
    p2(*xp+p3());
}
```

将其保存为t2.c文件

使用`gcc -c t2.c`生成可重定位的目标文件

使用`objdump -D t2.o > m.txt`反汇编并保存在m.txt内。在m.txt查看.text和.data

```
Disassembly of section .text:

00000000 <p2>:
   0:   55                      push   %ebp
   1:   89 e5                   mov    %esp,%ebp
   3:   5d                      pop    %ebp
   4:   c3                      ret

00000005 <p1>:
   5:   55                      push   %ebp
   6:   89 e5                   mov    %esp,%ebp
   8:   53                      push   %ebx
   9:   83 ec 14                sub    $0x14,%esp
   c:   a1 00 00 00 00          mov    0x0,%eax
  11:   8b 18                   mov    (%eax),%ebx
  13:   e8 fc ff ff ff          call   14 <p1+0xf>
  18:   01 d8                   add    %ebx,%eax
  1a:   89 04 24                mov    %eax,(%esp)
  1d:   e8 fc ff ff ff          call   1e <p1+0x19>
  22:   83 c4 14                add    $0x14,%esp
  25:   5b                      pop    %ebx
  26:   5d                      pop    %ebp
  27:   c3                      ret
```

```
Disassembly of section .data:

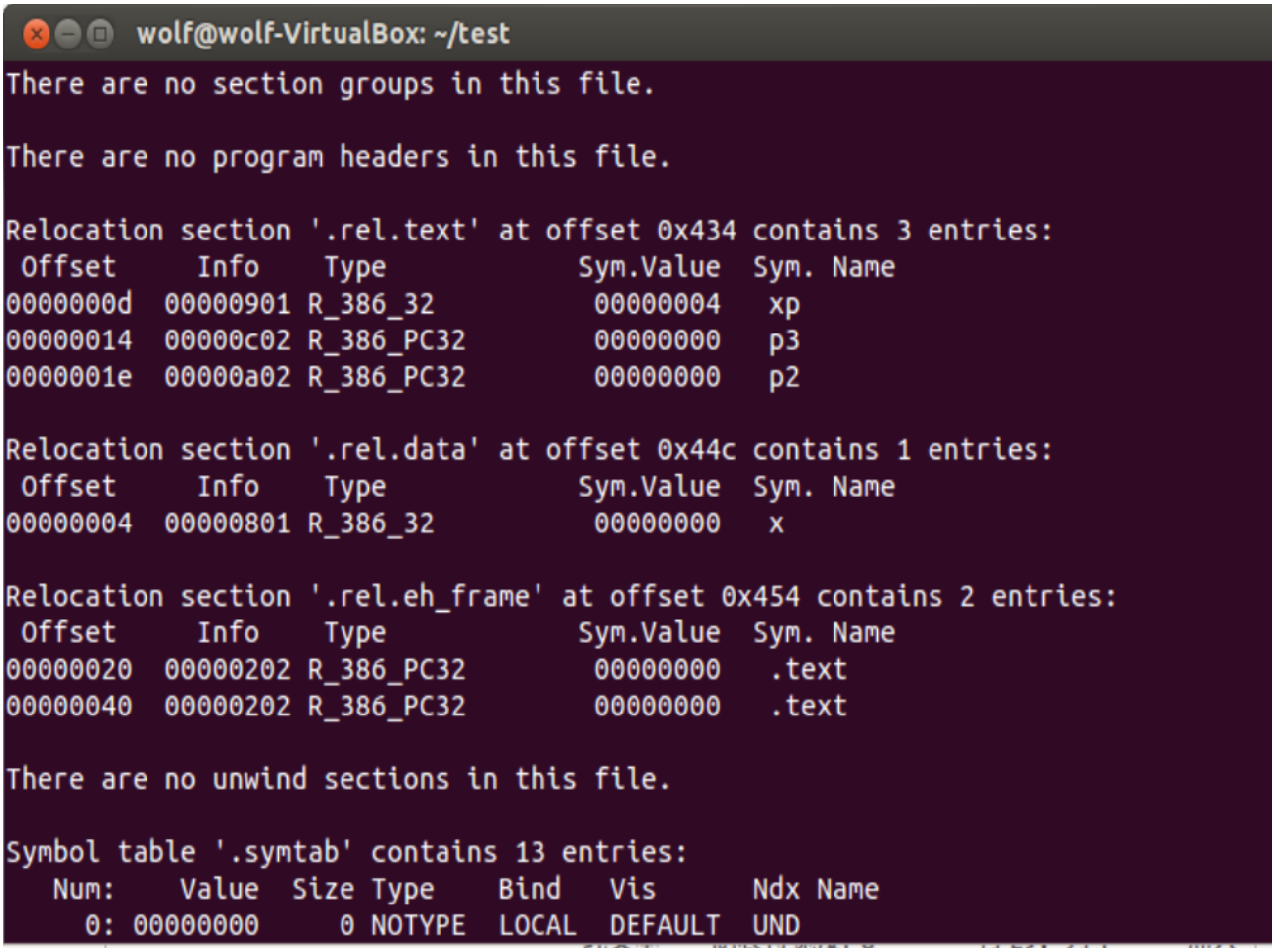
00000000 <x>:
   0:   01 00                   add    %eax,(%eax)
    ...

00000004 <xp>:
   4:   00 00                   add    %al,(%eax)
    ...
```

使用`readelf -a t2.o`可查看

```
Relocation section '.rel.text' at offset 0x434 contains 3 entries:
 Offset     Info    Type            Sym.Value  Sym. Name
0000000d  00000901 R_386_32          00000004   xp
00000014  00000c02 R_386_PC32        00000000   p3
0000001e  00000a02 R_386_PC32        00000000   p2

Relocation section '.rel.data' at offset 0x44c contains 1 entries:
 Offset     Info    Type            Sym.Value  Sym. Name
00000004  00000801 R_386_32          00000000   x
```



根据上述elf信息可推断

A.

节偏移	重定位类型	符号名字
0xd	绝对引用	xp
0x14	相对引用	p3
0x1e	相对引用	p2

B.

节偏移	重定位类型	符号名字
0x4	绝对引用	x

7.14

的重定位条目中的信息：节偏移、重定位类型和符号名字。
可以随意使用诸如 OBJDUMP 之类的工具来帮助你解答这个题目。

```
1   int relo3(int val)  {
2       switch (val) {
3       case 100:
4           return(val);
5       case 101:
6           return(val+1);
7       case 103: case 104:
8           return(val+3);
9       case 105:
10          return(val+5);
11      default:
12          return(val+6);
13      }
14  }
```

a）C 代码

```
1   00000000 <relo3>:
2      0:   55                      push   %ebp
3      1:   89 e5                   mov    %esp,%ebp
4      3:   8b 45 08                mov    0x8(%ebp),%eax
5      6:   8d 50 9c                lea    0xffffff9c(%eax),%edx
6      9:   83 fa 05                cmp    $0x5,%edx
7      c:   77 17                   ja     25 <relo3+0x25>
8      e:   ff 24 95 00 00 00 00    jmp    *0x0(,%edx,4)
9     15:   40                      inc    %eax
10    16:   eb 10                   jmp    28 <relo3+0x28>
11    18:   83 c0 03                add    $0x3,%eax
12    1b:   eb 0b                   jmp    28 <relo3+0x28>
13    1d:   8d 76 00                lea    0x0(%esi),%esi
14    20:   83 c0 05                add    $0x5,%eax
15    23:   eb 03                   jmp    28 <relo3+0x28>
16    25:   83 c0 06                add    $0x6,%eax
17    28:   89 ec                   mov    %ebp,%esp
18    2a:   5d                      pop    %ebp
19    2b:   c3                      ret
```

b）可重定位目标文件的 .text 节

```
    This is the jump table for the switch statement
1   0000 28000000 15000000 25000000 18000000 4 words at offsets 0x0,0x4,0x8, and 0xc
2   0010 18000000 20000000                   2 words at offsets 0x10 and 0x14
```

c）可重定位目标文件的 .rodata 节

图 7-21　练习题 7.14 的示例代码

请注意：与前一题要求一致，请在你自己的机器上运行题干中的代码（需要进行代码补齐），并根据你自己机器上观察到的可执行文件和可重定位目标文件的内容进行相关作答，需要展示解答过程。

我的答案：

题目代码如下：

```
int relo3(int val){
    switch(val){
        case 100:
            return (val);
        case 101:
            return (val+1);
        case 103: case 104:
            return (val+3);
        case 105:
            return (val+5);
        default:
            return (val+6);
    }
}
```

保存为t3.c文件

使用`gcc -c t3.c`生成可重定位的目标文件

使用`objdump -D t3.o > m.txt`反汇编并保存在m.txt内。在m.txt查看.text和.data

截图如下

```
wolf@wolf-VirtualBox: ~/test
wolf@wolf-VirtualBox:~$ cd testr
bash: cd: testr: 没有那个文件或目录
wolf@wolf-VirtualBox:~$ clear
wolf@wolf-VirtualBox:~$ cd test
wolf@wolf-VirtualBox:~/test$ gcc -c t3.c
wolf@wolf-VirtualBox:~/test$ objdump -D t3.o > m.txt
wolf@wolf-VirtualBox:~/test$ readelf -a t3.o
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Intel 80386
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          336 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
```

查看m.txt中.text与.rodata段

```
Disassembly of section .text:

00000000 <relo3>:
   0:   55                      push   %ebp
   1:   89 e5                   mov    %esp,%ebp
   3:   8b 45 08                mov    0x8(%ebp),%eax
   6:   83 e8 64                sub    $0x64,%eax
   9:   83 f8 05                cmp    $0x5,%eax
   c:   77 26                   ja     34 <relo3+0x34>
   e:   8b 04 85 00 00 00 00    mov    0x0(,%eax,4),%eax
  15:   ff e0                   jmp    *%eax
  17:   8b 45 08                mov    0x8(%ebp),%eax
  1a:   eb 1e                   jmp    3a <relo3+0x3a>
  1c:   8b 45 08                mov    0x8(%ebp),%eax
  1f:   83 c0 01                add    $0x1,%eax
  22:   eb 16                   jmp    3a <relo3+0x3a>
  24:   8b 45 08                mov    0x8(%ebp),%eax
  27:   83 c0 03                add    $0x3,%eax
  2a:   eb 0e                   jmp    3a <relo3+0x3a>
  2c:   8b 45 08                mov    0x8(%ebp),%eax
```

```
  2f:   83 c0 05                add    $0x5,%eax
  32:   eb 06                   jmp    3a <relo3+0x3a>
  34:   8b 45 08                mov    0x8(%ebp),%eax
  37:   83 c0 06                add    $0x6,%eax
  3a:   5d                      pop    %ebp
  3b:   c3                      ret
```

```
00000000 <.rodata>:
   0:   17                      pop    %ss
   1:   00 00                   add    %al,(%eax)
   3:   00 1c 00                add    %bl,(%eax,%eax,1)
   6:   00 00                   add    %al,(%eax)
   8:   34 00                   xor    $0x0,%al
   a:   00 00                   add    %al,(%eax)
   c:   24 00                   and    $0x0,%al
   e:   00 00                   add    %al,(%eax)
  10:   24 00                   and    $0x0,%al
  12:   00 00                   add    %al,(%eax)
  14:   2c 00                   sub    $0x0,%al
    ...
```

使用`readelf -a t3.o`可查看

```
Relocation section '.rel.text' at offset 0x42c contains 1 entries:
 Offset     Info    Type            Sym.Value  Sym. Name
00000011  00000501 R_386_32          00000000   .rodata

Relocation section '.rel.rodata' at offset 0x434 contains 6
entries:
 Offset     Info    Type            Sym.Value  Sym. Name
00000000  00000201 R_386_32          00000000   .text
00000004  00000201 R_386_32          00000000   .text
00000008  00000201 R_386_32          00000000   .text
0000000c  00000201 R_386_32          00000000   .text
00000010  00000201 R_386_32          00000000   .text
00000014  00000201 R_386_32          00000000   .text
```

根据上述elf信息可推断

A.

节偏移	重定位类型	符号名字
0x11	绝对引用	.rodata

B.

节偏移	重定位类型	符号名字
0x0	绝对引用	.text
0x4	绝对引用	.text
0x8	绝对引用	.text
0xc	绝对引用	.text
0x10	绝对引用	.text
0x14	绝对引用	.text

第8章

8.23

**8.23 你的一个同事想要使用信号来让一个父进程对发生在一个子进程中的事件计数。其思想是每次发生一个事件时，通过向父进程发送一个信号来通知它，并且让父进程的信号处理程序对一个全局变量 counter 加一，在子进程终止之后，父进程就可以检查这个变量。然而，当他在系统上运行图 8-41 中的测试程序时，发现当父进程调用 printf 时，counter 的值总是 2，即使子进程向父进程发送了 5 个信号。他很困惑，向你寻求帮助。你能解释这个程序有什么错误吗？

code/ecf/counterprob.c

```
1   #include "csapp.h"
2
3   int counter = 0;
4
5   void handler(int sig)
6   {
7       counter++;
8       sleep(1); /* Do some work in the handler */
9       return;
10  }
11
12  int main()
13  {
14      int i;
15
16      Signal(SIGUSR2, handler);
17
18      if (Fork() == 0) {  /* Child */
19          for (i = 0; i < 5; i++) {
20              Kill(getppid(), SIGUSR2);
21              printf("sent SIGUSR2 to parent\n");
22          }
23          exit(0);
24      }
25
26      Wait(NULL);
27      printf("counter=%d\n", counter);
28      exit(0);
29  }
```

code/ecf/counterprob.c

图 8-41　家庭作业 8.23 中引用的计数器程序

当父进程接收到第一个SIGUSR2信号之后，父进程进入信号处理例程。与此同时，由于隐式阻塞机制，剩余4个该类信号SIGUSR2被阻塞，并没有被接收。其中只有1个被视为待处理信号而被保存，其余的3个被丢弃。因此最终只有2个被处理，counter最终结果为2。

这是书上对于信号的基础概念。简单来说就是同一时间同一信号最多处理一个，挂起一个，丢弃剩余全部。

8.20

**8.20 使用 execve 编写一个叫做 myls 的程序，该程序的行为和 /bin/ls 程序的一样。你的程序应该接受相同的命令行参数，解释同样的环境变量，并产生相同的输出。

ls 程序从 COLUMNS 环境变量中获得屏幕的宽度。如果没有设置 COLUMNS，那么 ls 会假设屏幕宽 80 列。因此，你可以通过把 COLUMNS 环境设置得小于 80，来检查你对环境变量的处理：

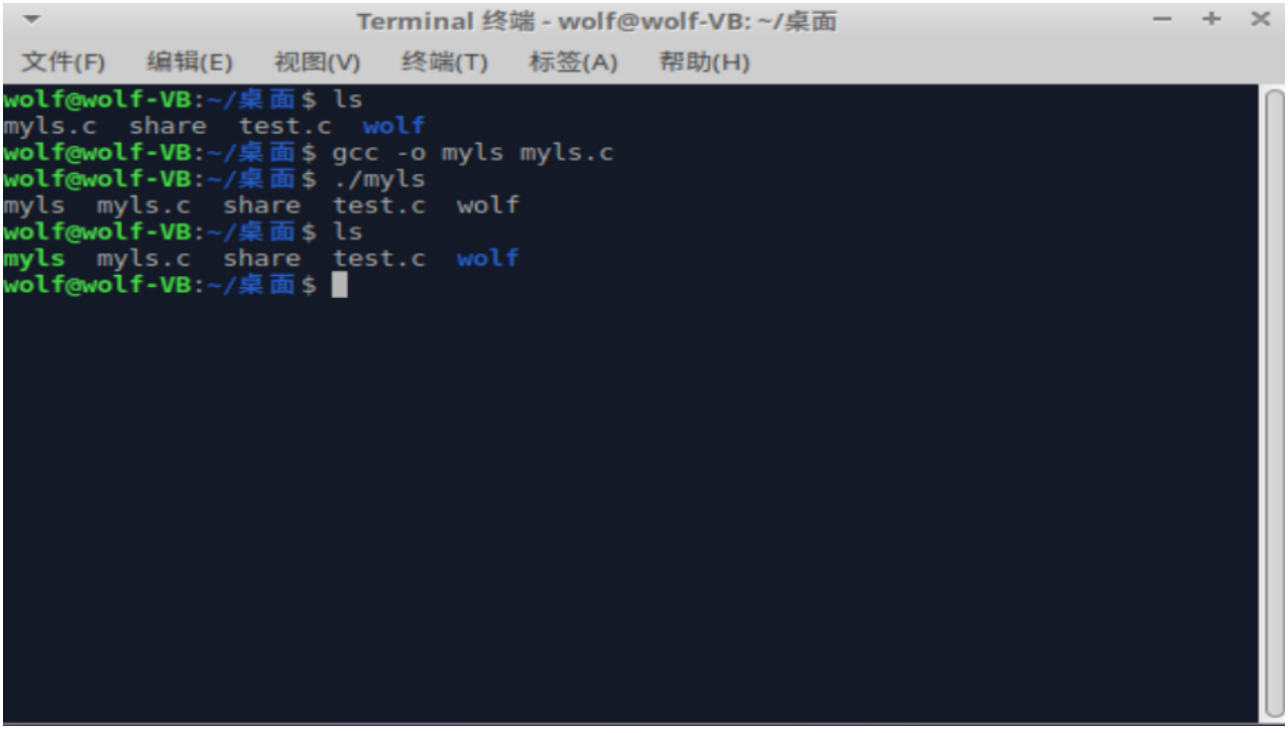
```
unix> setenv COLUMNS 40
unix> ./myls
 ...output is 40 columns wide
unix> unsetenv COLUMNS
unix> ./myls
 ...output is now 80 columns wide
```

代码如下：

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, const char *argv[], const char *envp[])
{
    if (execve("/bin/ls", argv, envp) < 0)
    {
        printf("/lib/ls not found.\n");
        return -1;
    }
    return 0;
}
```

测试结果：

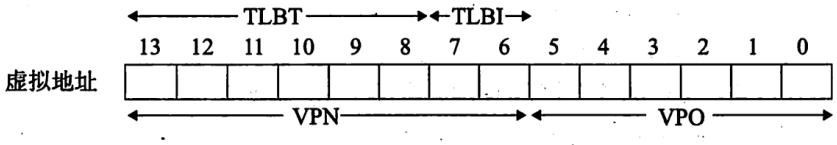


第9章

第一题

请完成《深入理解计算机系统》（第2版）P586~588,9.11~9.13，请体现你的完成过程。

示例内存系统如下：

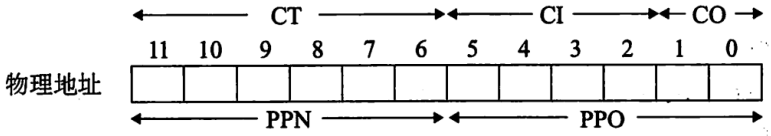


位	标记位	PPN	有效位	标记位	PPN	有效位	标记位	PPN	有效位	标记位	PPN	有效位
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

a）TLB：四组，16个条目，四路组相联

VPN	PPN	有效位	VPN	PPN	有效位
00	28	1	08	13	1
01	—	0	09	17	1
02	33	1	0A	09	1
03	02	1	0B	–	0
04	—	0	0C	–	0
05	16	1	0D	2D	1
06	—	0	0E	11	1
07	—	0	0F	0D	1

b）页表：只展示了前16个PTE



索引	标记位	有效位	块 0	块 1	块 2	块 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	08
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

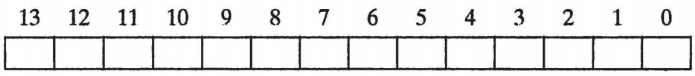
c）高速缓存：16 个组，4 字节的块，直接映射

9.11

*9.11 在下面的一系列问题中，你要展示 9.6.4 节中的示例存储器系统如何将虚拟地址翻译成物理地址，以及如何访问缓存。对于给定的虚拟地址，请指出访问的 TLB 条目、物理地址，以及返回的缓存字节值。请指明是否 TLB 不命中，是否发生了缺页，是否发生了缓存不命中。如果有缓存不命中，对于“返回的缓存字节”用“–”来表示。如果有缺页，对于“PPN”用“–”来表示，而 C 部分和 D 部分就空着。

虚拟地址：0x027c

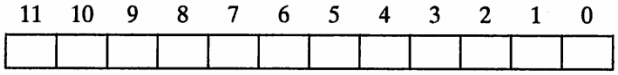
A. 虚拟地址格式



B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中？（是 / 否）	
缺页？（是 / 否）	
PPN	

C. 物理地址格式



D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中？（是 / 否）	
返回的缓存字节	

答案：

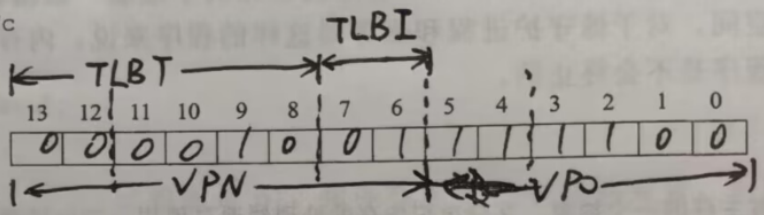
家庭作业

9421

9.11 在下面的一系列问题中，你要展示 9.6.4 节中的示例内存系统如何将虚拟地址翻译成物理地址，以及如何访问缓存。对于给定的虚拟地址，请指出访问的 TLB 条目、物理地址，以及返回的缓存字节值。请指明是否 TLB 不命中，是否发生了缺页，是否发生了缓存不命中。如果有缓存不命中，对于“返回的缓存字节”用“-”来表示。如果有缺页，对于“PPN”用“-”来表示，而 C 部分和 D 部分就空着。

虚拟地址： 0x027c

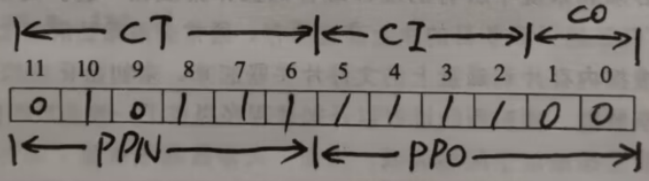
A. 虚拟地址格式



B. 地址翻译

参数	值
VPN	~~0x13~~ 0x9
TLB索引	0x1
TLB标记	0x2
TLB命中？（是/否）	否
缺页？（是/否）	是否
PPN	0x17

C. 物理地址格式



D. 物理地址引用

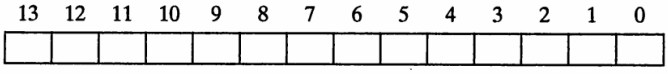
参数	值
字节偏移	0x0
缓存索引	0xf
缓存标记	0x17
缓存命中？（是/否）	否
返回的缓存字节	-

9.12

*9.12 对于下面的地址，重复习题 9.11：

虚拟地址：0x03a9

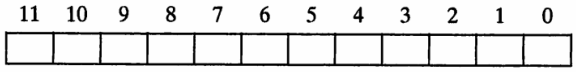
A. 虚拟地址格式



B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中？（是 / 否）	
缺页？（是 / 否）	
PPN	

C. 物理地址格式

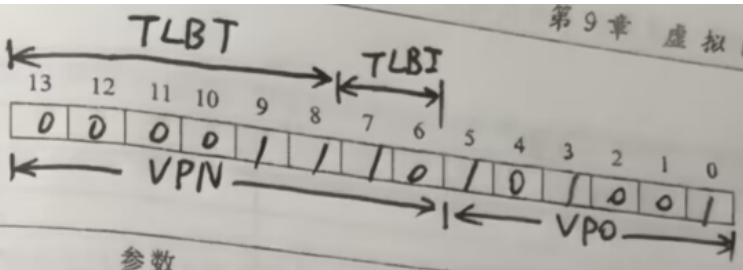


D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中？（是 / 否）	
返回的缓存字节	

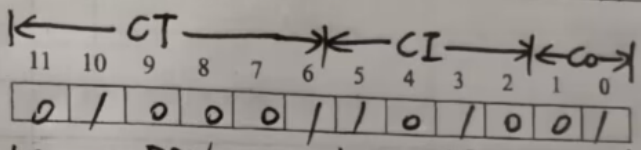
答案：

B. 地址翻译

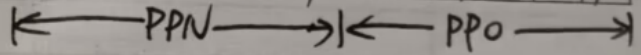


参数	值
VPN	0xe
TLB索引	0x2
TLB标记	0x3
TLB命中？（是/否）	否
缺页？（是/否）	否
PPN	0x11

C. 物理地址格式



D. 物理地址引用



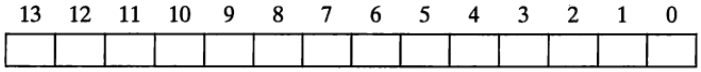
参数	值
字节偏移	0x1
缓存索引	0xa
缓存标记	0x11
缓存命中？（是/否）	否
返回的缓存字节	—

9.13

*9.13 对于下面的地址，重复习题 9.11：

虚拟地址：0x0040

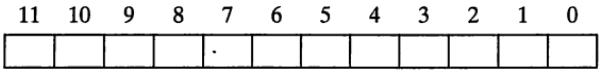
A. 虚拟地址格式



B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中？（是 / 否）	
缺页？（是 / 否）	
PPN	

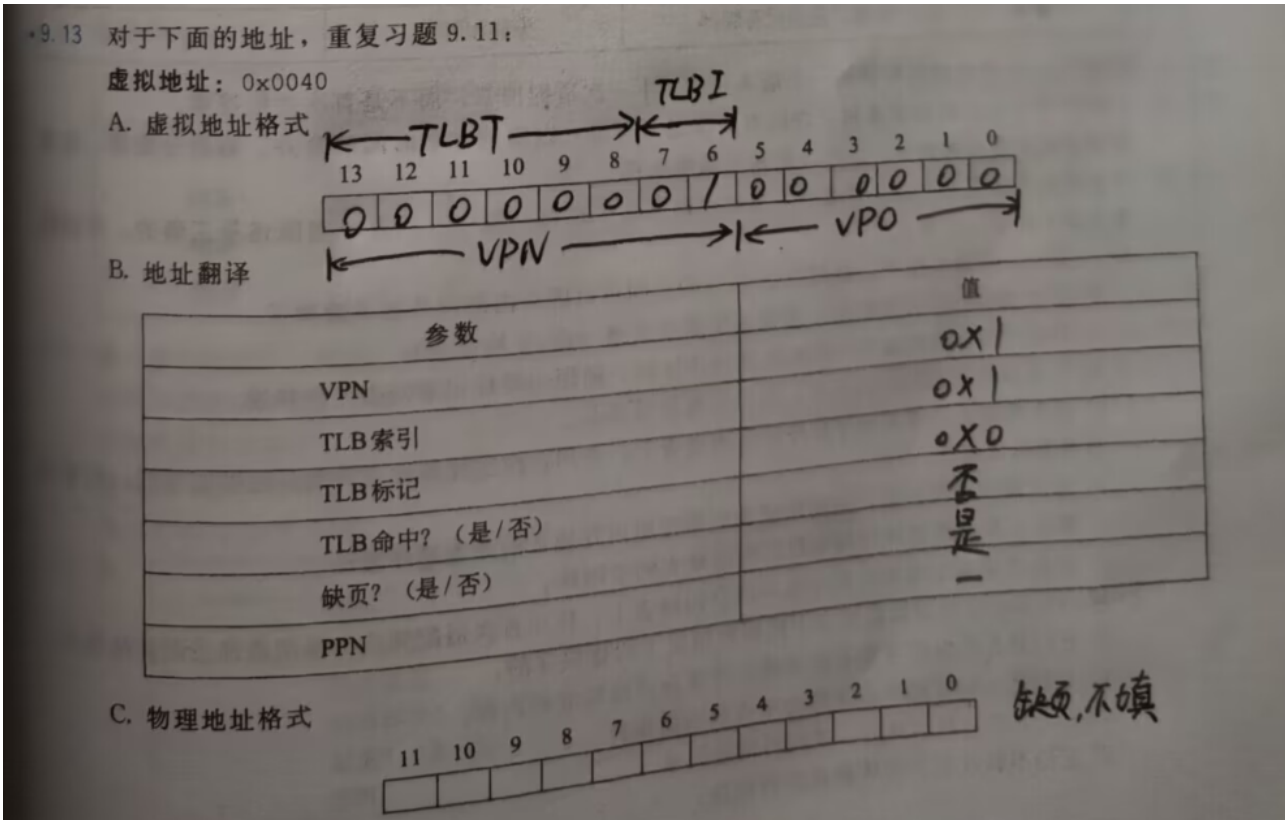
C. 物理地址格式

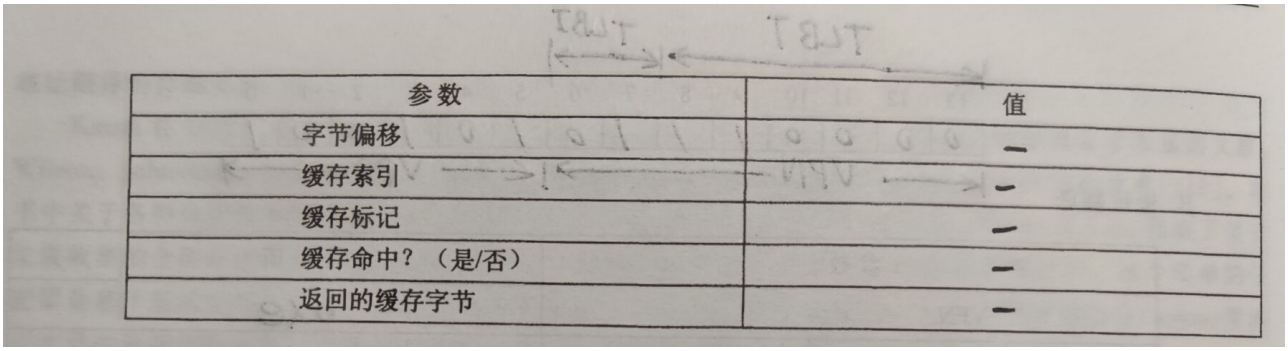


D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中？（是 / 否）	
返回的缓存字节	

答案：





第二题

设若有一个输入文件hello.txt，由字符串“Hello，World！\n”组成，编写一个C程序，使用mmap将该txt文件的内容修改为“Hello， HNU！\n”。

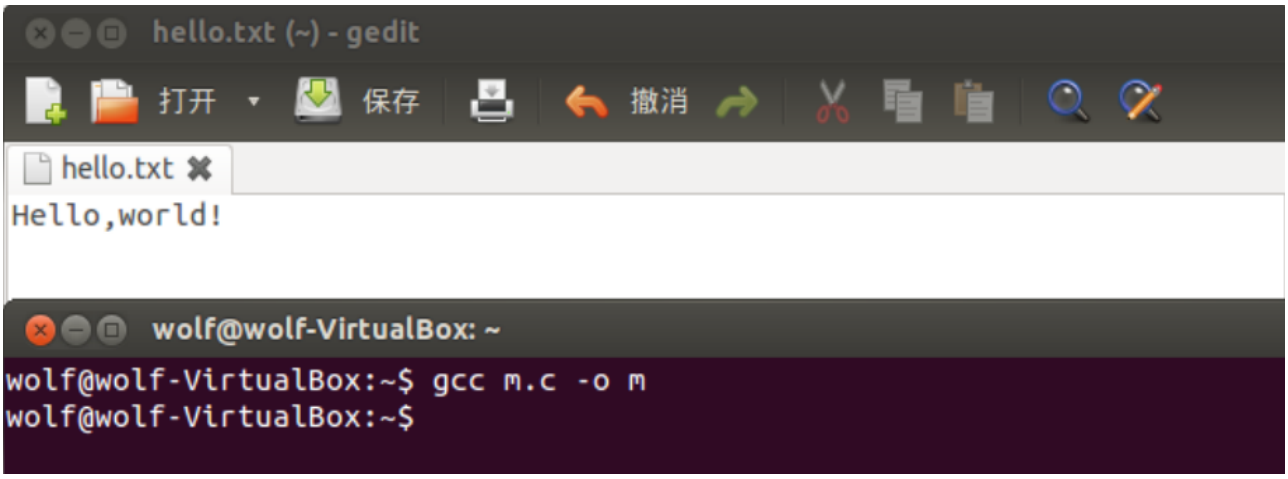
代码如下：

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

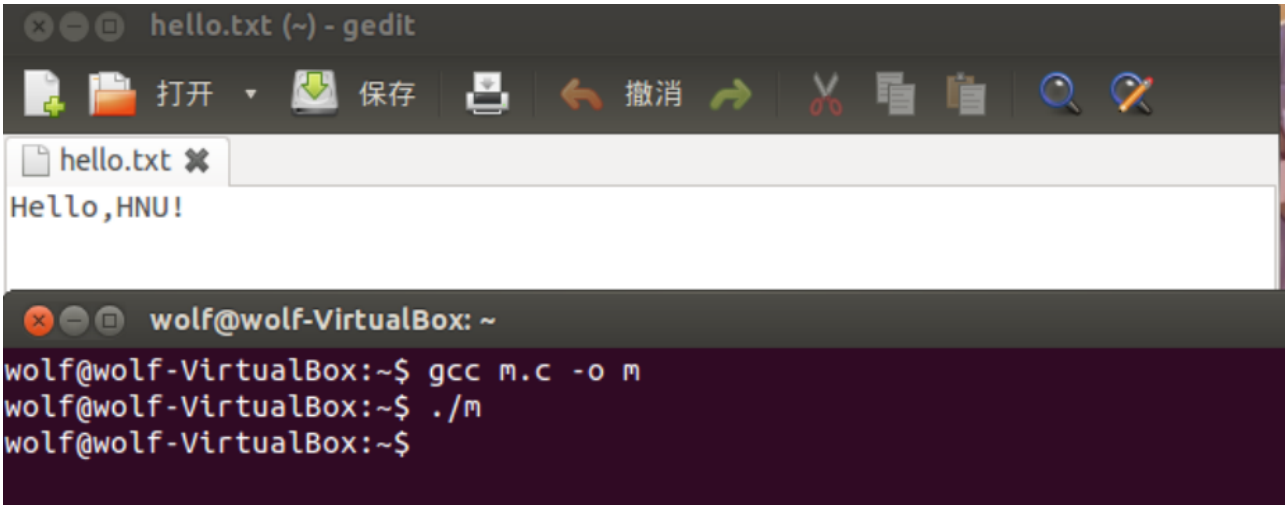
```c
#include <sys/mman.h>

int main()
{
    int fd = open("hello.txt",O_RDWR,0);
    char *start = mmap(NULL,1,PROT_WRITE,MAP_SHARED,fd,0);
    ftruncate(fd,11);
    close(fd);
    if(start == MAP_FAILED)
        printf("error!\n");
    else
    {
        char* newstr="Hello,HNU!\n";
        char *temp = start;
        int i=0;
        for (; i<11;temp++,i++)
            *temp=newstr[i];
        munmap(start,1);
        return 0;
    }
}
```

使用程序前：



使用程序后：


hello.txt (~) - gedit
打开
保存
撤消
hello.txt
Hello,HNU!
wolf@wolf-VirtualBox: ~
wolf@wolf-VirtualBox:~$ gcc m.c -o m
wolf@wolf-VirtualBox:~$ ./m
wolf@wolf-VirtualBox:~$